

How to Set the Access Password - REST and MQTT

Protected Mode requires the target tag to already have a valid 32-bit access password. The password is stored in **words 2-3 of the Gen2 `RESERVED` bank**:

Words	Contents
0-1	Kill password
2-3	Access password <- set this

Setting it uses `set_mode` access operations, not `impinjGen2X`.

How the calls work

Each `set_mode` below must be followed by an inventory:

```
set_mode -> start -> wait ~12 s -> stop
```

The access operation executes **when the tag is singulated**, not when the command returns. Results appear in the **tag data stream** as `accessResults`, never in the command response.

Action	REST	MQTT
Set mode	PUT /cloud/mode	{"command": "set_mode", "payload": { ... }}
Start	PUT /cloud/start with {}	{"command": "start", "payload": {}}
Stop	PUT /cloud/stop	{"command": "stop", "payload": {}}

Without `start`, `set_mode` returns `200`, the radio stays inactive, and nothing happens.

Step 1 - Read the current password

Many tags ship with a password already programmed. Read words 0-3 first.

REST - PUT /cloud/mode

```
{
  "type": "CUSTOM",
  "antennas": [1],
  "transmitPower": 30,
  "filter": {
    "value": "e28011b0a5050076c4d7530a",
    "match": "prefix",
    "operation": "include"
  },
  "accesses": [
    {
      "type": "READ",
      "config": {
        "membank": "RESERVED",
        "wordPointer": 0,
        "wordCount": 4
      }
    }
  ]
}
```

MQTT - set_mode

```
{
  "command": "set_mode",
  "payload": {
    "type": "CUSTOM",
    "antennas": [1],
    "transmitPower": 30,
    "filter": {
      "value": "e28011b0a5050076c4d7530a",
      "match": "prefix",
      "operation": "include"
    },
    "accesses": [
      {
        "type": "READ",
        "config": {
          "membank": "RESERVED",
          "wordPointer": 0,
          "wordCount": 4
        }
      }
    ]
  }
}
```

Then **start** -> wait -> **stop**. The result is 16 hex characters covering words 0-3:

```
{"data": {"idHex": "e28011b0a5050076c4d7530a",
  "accessResults": ["1234876512348765"]}}
```

```
1234876512348765
-----
kill password    (words 0-1)
ACCESS password  (words 2-3)
```

Read back	Next
0000000000000000	No password set - continue to step 2
Any other value	A password exists - use the last 8 characters, or continue to step 2 to change it

Step 2 - Write the password

Write 8 hex characters to **wordPointer**: 2. The same call assigns a first password or replaces an existing one.

REST - PUT /cloud/mode

```
{
  "type": "CUSTOM",
  "antennas": [1],
  "transmitPower": 30,
  "filter": {
    "value": "e28011b0a5050076c4d7530a",
    "match": "prefix",
    "operation": "include"
  },
  "accesses": [
    {
      "type": "WRITE",
      "config": {
        "membank": "RESERVED",
        "wordPointer": 2,
        "data": "A1B2C3D4"
      }
    }
  ]
}
```

MQTT - set_mode

```
{
  "command": "set_mode",
  "payload": {
    "type": "CUSTOM",
    "antennas": [1],
    "transmitPower": 30,
    "filter": {
      "value": "e28011b0a5050076c4d7530a",
      "match": "prefix",
      "operation": "include"
    },
    "accesses": [
      {
        "type": "WRITE",
        "config": {
          "membank": "RESERVED",
          "wordPointer": 2,
          "data": "A1B2C3D4"
        }
      }
    ]
  }
}
```

Then **start** -> wait -> **stop**. Each singulation produces a result:

```
{"data": {"idHex": "e28011b0a5050076c4d7530a", "accessResults": ["SUCCESS"]}}
```

Occasional **CRC error in tag response** or **Tag did not respond** entries are normal RF behaviour at the edge of range. At least one **SUCCESS** confirms the write.

Include the `filter`. Without it the password is written to **every tag in the field**.

Do not use `00000000`. On a Gen2 tag that means "no password set", so the tag cannot be protected. The request is accepted without error.

Step 3 - Read it back

Repeat step 1. The last 8 characters must match what was written:

```
{"data": {"idHex": "e28011b0a5050076c4d7530a",  
  "accessResults": ["00000000a1b2c3d4"]}}
```

```
00000000a1b2c3d4  
----- ACCESS password = a1b2c3d4
```

The password is now set and the tag is ready for Protected Mode.

Using the password

REST - PUT /cloud/impinjGen2X

```
{  
  "tagProtect": {  
    "action": "enableTagProtection",  
    "password": "A1B2C3D4",  
    "tagID": "e28011b0a5050076c4d7530a"  
  }  
}
```

MQTT - set_impinjGen2X

```
{  
  "command": "set_impinjGen2X",  
  "payload": {  
    "tagProtect": {  
      "action": "enableTagProtection",  
      "password": "A1B2C3D4",  
      "tagID": "e28011b0a5050076c4d7530a"  
    }  
  }  
}
```

Apply it on the next inventory start:

REST	MQTT
PUT /cloud/start with {"applyImpinjGen2X": true}	{"command": "start", "payload": {"applyImpinjGen2X": true}}

The password must match the one stored on the tag. A mismatch returns **200 OK** and leaves the tag unprotected, with no error.

Password requirements

Requirement	Value
Length	Exactly 8 hexadecimal characters (32-bit)
Characters	0-9, A-F
Case	Not significant - a1b2c3d4 = A1B2C3D4

Not permitted	00000000 - denotes "no password" on a Gen2 tag
---------------	--

Full call sequence

Step 1	set_mode	READ	RESERVED	wordPointer 0, wordCount 4
	start	->	read current password from tag stream	-> stop
Step 2	set_mode	WRITE	RESERVED	wordPointer 2, data <8 hex>
	start	->	confirm accessResults ["SUCCESS"]	-> stop
Step 3	set_mode	READ	RESERVED	wordPointer 0, wordCount 4
	start	->	confirm the new password is stored	-> stop

Purpose	REST	MQTT
Read / write tag memory	PUT /cloud/mode	set_mode
Start inventory	PUT /cloud/start	start
Stop inventory	PUT /cloud/stop	stop
Configure Protected Mode	PUT /cloud/impinjGen2X	set_impinjGen2X

Verified on FXR60 reader application 5.0.7 and FXR90 reader application 5.0.4 with Impinj Monza 4i tags.